

Variant Calling - Identifying SNPs and INDELs in Bacterial Populations

Authors: Palash Ghosh¹, Amanpreet Kaur¹ and Neha Potnis¹

ORCIDs: Palash Ghosh- 0009-0005-8146-2550

Amanpreet Kaur: 0009-0005-9467-5424

Affiliation: ¹Department of Entomology and Plant Pathology, Auburn University, Auburn, AL 36830.

Introduction

The field of genome sequencing has been transformed by advances in next-generation sequencing techniques. It has become easier to sequence DNA, and this technological progress has enabled the collection of large datasets, greatly enhancing our understanding of biological processes, disease outbreaks, and genetic research. Many of these studies are possible thanks to the wider use of variant calling in genomics, which also provides greater statistical power for biological interpretations. Read mapping is at the core of many genomic analysis workflows. The ability to accurately map sequencing reads to a reference genome is fundamental to downstream analyses including Association Studies (for association of genotypes to phenotypic traits): Genome-Wide-Association-Studies (GWAS) (Newberry et al. 2023), QTL mapping (Shu et al. 2021) and Marker discovery (Bhat et al. 2016); for Breeding and agricultural application (using variants for selection): Genomic selection and Trait prediction; Evolutionary Studies (tracing the genomic changes of specific organism over time): Phylogenetics (evolutionary relationships among different groups/lineages) (Newberry et al. 2023), Population genomics (structure and differentiation), Selection analysis (detection of selection pressures) (Guttman et al. 2006); Epidemiology & Disease Research: Outbreak tracing/molecular epidemiology (Abrahamian et al. 2019); identification of virulence genes and resistance mutations; clinical & medical genetics: Germline variant calling (rare/hereditary diagnosis) (AIDubayan et al. 2020); somatic variant calling (cancer genomics or target tumor therapies); Comparative Genomics: Diversity analyses (pan-genomics analysis), Structural variants (indels, duplications), Genome comparisons; Functional genomics: Expression quantitative Trait Loci (eQTL) mapping (Holloway et al. 2011); allele specific expression analysis (combining RNA-seq data with variant calling) (Bollas et al. 2025).

The goal of this paper is to present variant calling workflow (Part 1A) which is at the core of many of the above-mentioned applications and provide directions to one of the applications, to determine population structure (Part1B).

Part 1A. Identify variants in bacterial genomes

The primary objective of this exercise is to introduce the basic steps in variant calling approach using a simple, real-world example. We will analyze a small set of paired-end whole genome sequencing datasets from field isolates of *Xanthomonas perforans* collected over a period spanning from 1991 to the present. Using a 1991 isolate as a reference genome, we will identify and characterize sequence variations, including single-nucleotide polymorphisms (SNPs) and insertions/deletions (INDELs), that have accumulated in more recent isolates. This exercise provides hands-on experience with variant detection and illustrates how bacterial genomes can change over time in response to field selection pressures.

Primary Goal

Gain hands-on experience with a standard variant calling workflow by performing reference-based genome alignment, SNP/INDEL detection, and visualization of genome-wide sequence variation.

Learning Objectives

Upon completing this exercise, students will be able to:

- Importance of read mapping as a foundational step in genomic analysis and variant detection
- Execute complete read mapping pipelines from raw FASTQ data through BAM file generation
- Perform quality control steps including duplicate removal to improve variant calling accuracy
- Call variants using state-of-the-art tools (samtools/bcftools) and apply evidence-based filtering
- Interpret VCF files and understand variant annotations (quality scores, depth, allele frequency)
- Distinguish between true biological variants and artifacts, and filter accordingly
- Analyze temporal patterns of genomic change across field-collected isolates.

Duration

This exercise is designed to be completed in ~20 min.

Materials & Settings

- Laptop or workstation with access to a high-performance computing (HPC) cluster
- Terminal or command-line interface with bash shell
- Minimum 16-20 GB RAM for large alignment files
- Internet access for downloading tools and reference genomes
- Text editor or IDE (optional, for viewing/editing scripts)

Software & Tools

- BWA (v0.7.17+) - Read alignment
- SAMtools (v1.2+) - SAM/BAM file manipulation
- BCFtools (v1.9+) - Variant calling
- GATK (v4.0+) - Variant filtration

- VCFtools (v0.1.14+) - VCF file processing
- Picard (v1.119+) - PCR duplicate removal
- IGV (Integrative Genomics Viewer) - Visualization

Genomic Data for the analysis

Paired-end FASTQ sequencing data from *Xanthomonas perforans* isolates with NCBI accession numbers: SAMN16476190, SAMN33607572, SAMN33607647, SAMN33607652, SAMN33607656, and SAMN33607657 (designated as reference genome). All data should be pre-processed (quality trimming and adapter removal) before beginning this exercise.

Methods

Note 1: It is recommended to direct downloads through the website to ensure the correct species are being downloaded. Sometimes, if there are multiple pathovars within a species, unintended genomes may be downloaded when using automated command-line tools.

Note 2: We are including the HPC-based command lines to make it easier for users to understand the purpose of each command. We have a separate section for Windows and Mac using the offline system (Docker) because the HPC systems are unavailable for the workshop purpose; all the commands are available on our GitHub (<https://github.com/Potnislabs/APS-Plant-Health-Workshop-2026>).

Do you want to provide your single code that combines all steps? Say that this entire code takes 15 minutes on HPC with 15 cores and xxx memory to run for variant calling for five strains.

Then you can say that this code will perform following steps. These steps are mentioned below to explain line by line code, rationale for all the options, what input files are and what output files will get generated in each step.

STEP 1: Reference Genome Indexing

Why This Step: Creating an index file of the reference sequence allows which allows to search for alignment positions without checking the entire genome. This dramatically reduces computation time.

What We Do: We create two types of indices:

- BWA index (Burrows-Wheeler aligner index): Used by the BWA aligner to enable efficient alignment. BWA uses the BWT for memory-efficient indexing of large genomes.
- FASTA index (.fai): A tab-delimited file containing genome coordinates, required by GATK and other downstream tools for random-access reading of genomic regions.

Commands: See detailed commands in the full protocol documentation.

Command	Description
<pre>#!/bin/bash # variant calling against Xp91-118 reference # ref = Xp91-118.fasta module load bwa module load samtools/1.2 module load bcftools module load gatk module load vcftools</pre>	Load Modules exactly as required by the protocol
<pre># picard - download once, comment out after first run #wget http://sourceforge.net/projects/picard/files/picard- tools/1.119/picard-tools-1.119.zip #unzip picard-tools-1.119.zip</pre>	Download Picard if it doesn't exist
<pre>bwa index Xp91-118.fasta samtools faidx Xp91-118.fasta java -jar ./picard-tools-1.119/CreateSequenceDictionary.jar R=Xp91- 118.fasta O=Xp91-118.dict</pre>	bwa needs an index of the reference so it can find short reads inside it quickly. The faidx index is a separate thing that gatk needs later, and the .dict from picard is also for gatk. all three only need to be made once per reference

STEP 2: Read Mapping with BWA-MEM

Why This Step: Mapping identifies the genomic location of each sequencing read by aligning it to the reference genome. This establishes the positional context required for variant calling.

Algorithm Choice—BWA-MEM: We chose BWA-MEM because:

- Optimized for high-quality Illumina reads (70–1 Mb in length)
- Faster and more accurate than older BWA-backtrack algorithm
- Supports split alignments (useful for detecting structural variants)
- Superior to Bowtie2 for this application (Bowtie2 is better for very short reads <50 bp)

What We Do: Read paired-end FASTQ files (forward and reverse reads) for each field isolate and align both to the 1991 reference genome. The output is a SAM (Sequence Alignment/Map) file which is describing:

- Read name and sequence
- Alignment reference name and position
- Mapping quality (MAPQ)
- CIGAR string (details of matches, mismatches, indels)

Commands: See detailed commands in the full protocol documentation.

Command	Description

Command	Description
bwa mem -t 4 -M Xp91-118.fasta sample_1.fq sample_2.fq > mapped_bwa.sam	align the fwd and rev reads to the reference with bwa-mem. We are using mem because reads are >70bp is to backtrack for the shorter stuff. -M flags secondary hits so picard and gatk don't get stuck on them. output is a sam file.

STEP 3: SAM-to-BAM Conversion & Sorting

Why This Step: SAM files are human-readable but computationally inefficient. BAM (Binary Alignment/Map) is the compressed and smaller and far faster for computation.

What We Do: Convert SAM to BAM using samtools view -Sb, then sort the BAM file by genomic position using samtools sort.

Command	Description
samtools view -Sb mapped_bwa.sam > mapped_bwa.bam samtools sort -O bam -o mapped_bwa_sort.bam mapped_bwa.bam -T PREFIX.nnnn.bam	sam is plain text and slow to parse, so convert to bam (-S in, -b out) and then sort by position on the reference. most downstream tools want a sorted bam.

STEP 4: Duplicate Removal

Why This Step: Illumina sequencing requires PCR amplification of library molecules. Over-amplification creates duplicate reads i.e., multiple reads originating from the same DNA fragment. This inflates apparent variant support and increases false positive variant calls.

What We Do: Use Picard MarkDuplicates to:

- Identify reads with identical start coordinates (hallmark of PCR duplicates)
- Remove redundant copies, keeping the read with highest base quality
- Index the deduplicated BAM file for fast lookup

Command	Description
java -jar ./picard-tools-1.119/MarkDuplicates.jar REMOVE_DUPLICATES=true INPUT=mapped_bwa_sort.bam OUTPUT=mapped_bwa_sort_marked.bam METRICS_FILE=markduplicates_metrics CREATE_INDEX=true VALIDATION_STRINGENCY=LENIENT samtools index mapped_bwa_sort_marked.bam	illumina prep involves pcr so some reads are just copies of each other. picard finds reads with the same start position (both ends for PE), keeps the best one by base quality and drops the rest. This is important because pcr duplicates can make a sequencing error look like a real variant supported by lots of reads.

STEP 5: Variant Calling

Why This Step: Variant calling is the core analysis step. It identifies positions where each field isolate's consensus sequence differs from the 1991 reference, calling SNPs (single nucleotide polymorphisms) and indels.

What We Do: Use `samtools/bcftools mpileup + call` to:

- Generate a pileup: depth of reads at every genomic position
- Call variants: use Bayesian statistics to determine if alternate alleles are real mutations
- Output a VCF (Variant Call Format) file listing all detected variants with metadata

VCF Annotations:

- CHROM, POS: Reference sequence name and genomic position
- REF, ALT: Reference allele and detected alternate (mutant) allele
- QUAL: Phred-scaled quality score; higher = higher confidence
- DP: Read depth at this position (number of reads covering the variant)
- MQ: Root mean square mapping quality of supporting reads (0–60)
- FQ: Phred probability that all samples share the same allele. Negative = homozygous, Positive = heterozygous. In bacteria, heterozygosity may indicate duplicated genes or mixed populations.

Command	Description
<code>samtools mpileup -ug -f Xp91-118.fasta mapped_bwa_sort_marked.bam bcftools call -O v -v -c -o test.vcf</code>	<code>mpileup</code> walks the bam and reports the bases stacked at each position, <code>bcftools call</code> decides which of those are actually variants. <code>-v</code> keeps only variant sites, output is vcf.
<code>grep -v '#' test.vcf wc -l</code> <code>grep -v '#' test.vcf grep 'INDEL' wc -l</code>	how many variants total, and how many of those are indels

STEP 6: Variant Filtering

6.1 - Confidence Filtering with GATK Variant Filtration

Why This Step: Not all variant calls are equally reliable. Sequencing errors, mapping artifacts, and low-coverage regions can produce false positives. Filtering removes low-confidence variants based on quality metrics.

Filtering Criteria (in this exercise):

- DP > 15: At least 15 reads supporting the variant, which reduces noise from low-coverage positions

- MQ > 50: Reads have high mapping quality (well-aligned, unambiguous position)
- QUAL > 100: High confidence in variant assertion (Phred score > 100 $\approx$ 1 error in 10¹⁰)
- FQ < 0.025: Negative FQ suggests homozygous variant; for bacterial haploid genomes, true variants should be homozygous

6.2 - INDEL Removal with VCFtools

Why This Step: This exercise focuses on SNPs (single nucleotide polymorphisms). INDELs (insertions/deletions) are structurally different and require specialized calling algorithms for accuracy. By keeping only SNPs, we ensure a clean, interpretable variant set.

What We Do: Use VCFtools to extract only variants that passed the GATK filter and remove all INDEL records, producing a final SNP-only VCF file.

Command	Description
<code>gatk VariantFiltration -R Xp91-118.fasta -O test.filter_gatk.vcf --variant test.vcf --filter-expression "FQ < 0.025 && MQ > 50 && QUAL > 100 && DP > 15" --filter-name pass_filter</code>	Raw calls include a lot of junk, this tags positions that clear our cutoffs with pass_filter in the FILTER column, - DP>15 means at least 15 reads back the call, MQ>50 is decent mapping quality, QUAL>100 is a confident call, FQ is the homozygous/heterozygous score (matters here for spotting gene duplications). Note-gatk doesn't remove anything, it just labels.
<code>grep -c 'pass_filter' test.filter_gatk.vcf</code>	Check the tag actually got added and count how many passed
<code>vcftools --vcf test.filter_gatk.vcf --keep-filtered pass_filter --remove-indels --recode --recode-INFO-all --out test_filter_onlysnp</code>	Now actually pull out the ones that passed, and drop indels so we're left with snps only

STEP 7: Visualization & Interpretation

Why This Step: Visual inspection of variant calls, read alignments, and genomic information provides reliability in results and can show important patterns.

Tools & Resources:

- IGV (Integrative Genomics Viewer): Interactive visualization of BAM files, VCF variants, and genome annotations in a single view
- Load files: reference genome (.fasta, .gff), BAM alignments for each isolate, VCF variants, and navigate to suspected loci for manual curation and temporal pattern analysis

Expected results:

I asked Amanpreet to add to the code so we get output text file with last 15 lines. Once you do that, say more output.txt and you will see following result.

Screenshot of the results showing mutations and indels

```
After filtering, kept 1 out of 1 individuals
Outputting VCF file...
After filtering, kept 0 out of a possible 703 Sites
No data left for analysis!
Run Time = 0.00 seconds
Finished processing SAMN33607657
=====
FINAL MUTATION COUNTS (Compared to Reference)
=====
SAMN16476190 : 6872 total SNPs
SAMN33607652 : 3665 total SNPs
SAMN33607647 : 70 total SNPs
SAMN33607652 : 147 total SNPs
SAMN33607656 : 2029 total SNPs
SAMN33607657 : 0 total SNPs
=====
```

- SAMN33607657: 0 SNPs, 703 raw calls and none passed filtering. Effectively identical to the reference, almost certainly 91-118 itself. Useful as a negative control showing the pipeline isn't generating false positives.
- SAMN33607647 (70 variants) and SAMN33607652 (147 variants): These isolates show relatively few differences from the reference genome, indicating that they are closely related to the ancestral lineage. The detected variants likely represent mutations that accumulated gradually over time through genetic drift and adaptation.
- SAMN33607656 (2,029 variants), SAMN33607645 (3,665 variants), and SAMN16476190 (6,872 variants): These isolates exhibit substantially greater genomic divergence from the reference genome. The large number of variants suggests that they belong to distinct evolutionary lineages rather than representing direct descendants of the original 1991 population. The increasing variant counts reflect progressively greater genetic divergence from the ancestral strain.

Variant counts provide a measure of genetic relatedness to the reference genome. Isolates with few variants are closely related to the ancestral population, whereas isolates with thousands of variants represent more distantly related lineages that have undergone substantial evolutionary change.

The Integrative Genomics Viewer (IGV) is a free desktop application for interactive visualization of genomic data. It displays a reference genome as a coordinate axis and stacks alignments, annotations, and variant calls as tracks above it, viewable from whole genome scale down to a single base. IGV lets us visually confirm any SNP or indel which is the quickest way to separate true mutations from mapping and sequencing artifacts.

- Download IGV from <https://software.broadinstitute.org/software/igv/download> and open it.
- Genomes → Load Genome from File → pick Xp91-118.fasta. It's a custom genome, so it won't be in the dropdown list.

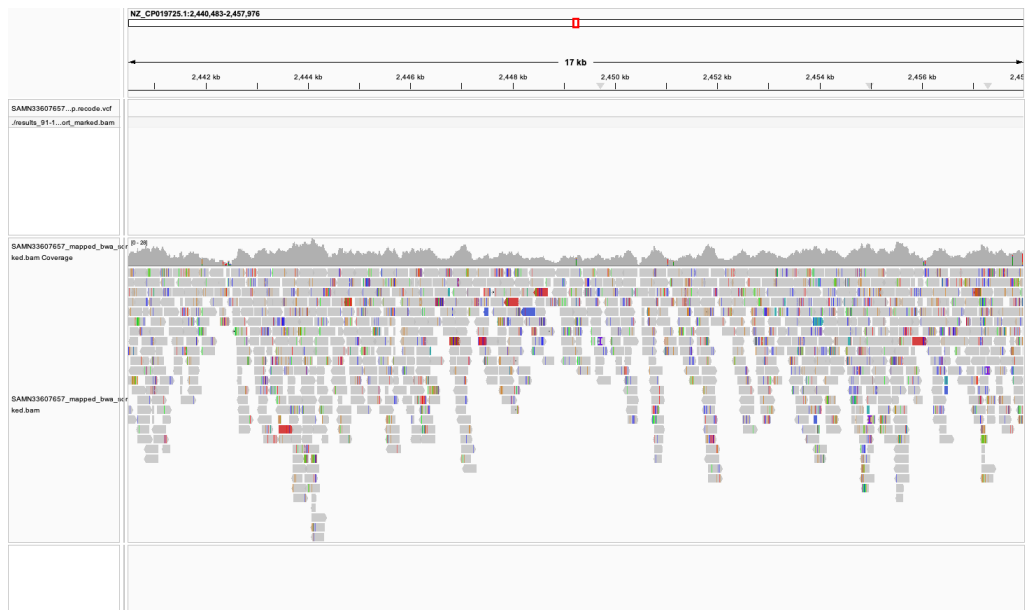
File → Load from File → Xp91-118.gff to get the gene annotation track. This is what lets you see which gene a SNP falls in.

- File → Load from File → mapped_bwa_sort_marked.bam. You'll get a coverage track and the read alignment track below it.

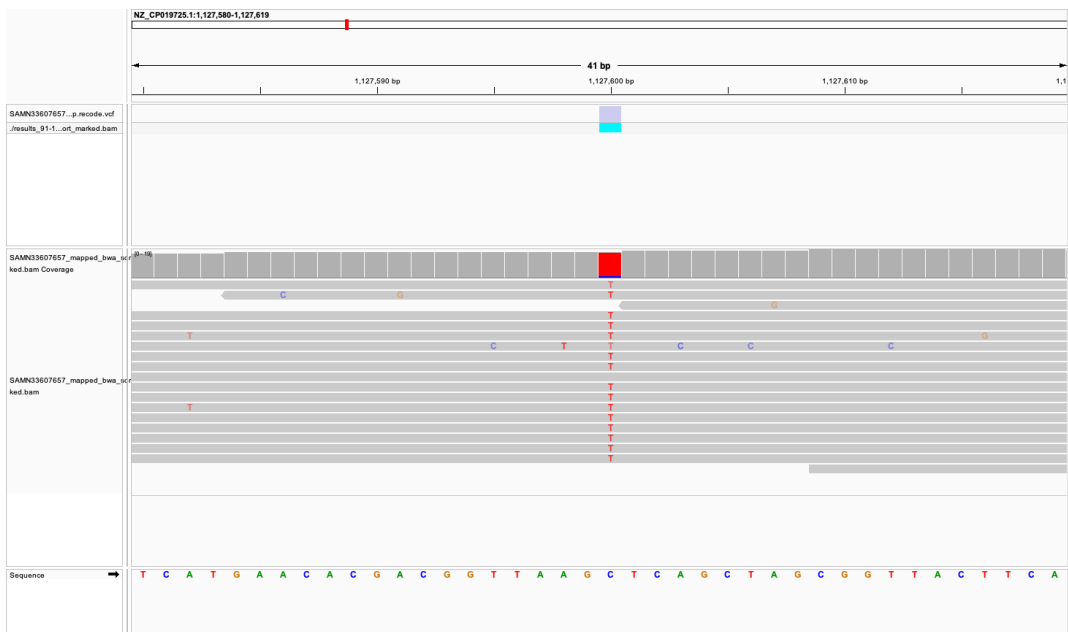
- File → Load from File → test_filter_onlysnp.recode.vcf for the filtered SNPs. Load test.filter_gatk.vcf too if you want to compare filtered against unfiltered.
- Pick the contig from the chromosome dropdown, then either zoom in or paste a position into the search box (e.g. NZ_CP019725.1:1,127,580-1,127,619). Reads won't render until you're below ~30 kb.

Things worth doing once you're there:

- Right-click the alignment track → **Sort by base** to line up reads carrying the alternate allele.
- Right-click → **Color alignments by** → **mapping quality** to spot the mismapped repeat regions.
- Hover a read to see its MAPQ; hover the coverage bar to get the exact ref/alt read counts at that position.
- **View** → **Preferences** → **Alignments** lets you raise the visibility threshold if you want to view a wider window.



IGV panel 1 (17 kb)-Coverage is continuous, but mismatches are scattered across reads at non-shared positions and the pile is ragged. That's mismapping, not variation likely a repetitive or paralogous region. The VCF track is empty here, which is correct: the $MQ > 50$ filter removed these.



IGV panel 2 (41 bp)-A clean homozygous SNP at ~1,127,600. Every read carry T where the reference has C, coverage goes solid red, and the variant appears in the filtered VCF. The isolated colored bases elsewhere are single-read sequencing errors and were correctly not called.

Suggestions for Determining Student Learning

To evaluate student understanding of read mapping and variant calling, consider the following assessment strategies:

- Variant count analysis: Have students compare the total number of SNPs called across different isolates and explain why variation exists
- Comparative analysis: Have students identify which genomic regions show the most variation across isolates and discuss what this reveal about evolutionary pressure
- Visual interpretation: Use IGV to display variant calls and read alignments, asking students to identify potential false positives or alignment artifacts

Part 1B. Population Structure Analyses of bacterial genomes

In Part 1A, we mapped raw sequencing reads from individual field isolates against the reference genome to detect SNPs and INDELs. While single-isolate variant calling is ideal for identifying local mutational changes in individual samples, understanding the broader evolutionary trajectory of a pathogen population required scaling these SNV profiles across multiple genomes. By moving from single -isolate mapping to population-wide core-genome alignment, we isolate the conserved regions shared across all strains and eliminate accessory regions. This clean, core-SNP matrix provides the high-resolution signal necessary to build robust evolutionary trees, trace outbreak lineages across time and geography, and uncover fine-scale population structure within field samples.

This Part 1B focuses on constructing high-resolution phylogenies of bacterial populations using core-genome SNP analysis. We will use publicly available *Xanthomonas perforans* genomes to demonstrate how reference-based alignment, SNP extraction, and hierarchical Bayesian population structure analysis can resolve fine-scale evolutionary relationships and define distinct genetic clusters (lineages) within a species. This workflow is directly transferable to other bacterial pathogens and enables epidemiological tracking, evolutionary reconstruction, and population subdivision analysis.

Primary Goal:

Transition from single-isolate variants to population-scale genomics by constructing core-genome single-nucleotide polymorphism alignments across field-isolated genomes to reconstruct high-resolution evolutionary phylogenies and delineate population/lineage clusters.

Learning Objectives:

Upon completion of this exercise, participants will be able to:

- Annotate bacterial reference genomes to generate GenBank-formatted reference scaffolds for downstream analyses.
- Construct core-genome alignments across multiple assembled genomes and extract segregating core SNP matrices
- Define groupings for bacterial isolates (Sequence Clusters) based on core SNPs in conserved regions.

Materials & Software:

- Prokka v1.14+ (genome annotation)
- Parsnp v1.5.6+ (core-genome alignment)
- HarvestTools v1.2+ (SNP extraction)
- R with packages: ape, rhierbaps, ggplot2, ggnewscale, ggtree, ggtreeExtra
- Complete *Xanthomonas perforans* genome sequences (NCBI GenBank)

Script:

Step a. Data Acquisition

First, download all genomes of the focal species from the NCBI website. It can be done either through the command line or via direct download from the website and then upload them to the High-Performance Computing (HPC) cluster.

***Note 1:** It is recommended to direct downloads through the website to ensure the correct species are being downloaded. Sometimes, if there are multiple pathovars within a species, unintended genomes may be downloaded when using automated command-line tools.*

***Note 2:** We are including the HPC-based command lines to make it easier for users to understand the purpose of each command. We have a separate section for Windows and Mac using the offline system (Docker) because the HPC systems are unavailable for the workshop purpose; all the commands are available on our GitHub (<https://github.com/Potnislabs/APS-Plant-Health-Workshop-2026>).*

Step b. Genome Annotation using Prokka

Before running an alignment, we need a reference genome in gbk format; hence we need to annotate the genome. This step uses Prokka (<https://github.com/tseemann/prokka>) (Seemann 2014) to rapidly annotate a bacterial genome.

Command	Description
<pre>#!/bin/bash module load anaconda/3-2020.02 prokka \ --prefix 91-118 \ --locustag 91-118 \ --increment 10 \ --outdir 91-118_Prokka \ --force \ --addgenes \ --genus Xanthomonas \ --gcode 11 \ GCF_000192045.2_ASM19204v3_genomic.fna</pre>	<pre># Run Prokka genome annotation # Load the Anaconda module to access the Prokka environment # prokka: Calls the Prokka software tool. # --prefix 91-118: Sets the prefix for all output files to "91-118". # --locustag 91-118: Sets the locus tag prefix for the annotated genes to "91-118". # --increment 10: adding a gap of 10 in the locus tag numbering by 10 for each successive gene. # --outdir 91-118_Prokka: Creates a directory named "91-118_Prokka" to store the results. # --force: Forces Prokka to overwrite the output directory if it already exists. # --addgenes: Instructs Prokka to add 'gene' features for each 'CDS' (coding sequence) found. # --genus Xanthomonas: Specifies the genus to improve database searching and naming. # --gcode 11: Uses genetic code 11, which is the standard genetic code for Bacteria and Archaea. # GCF_...genomic.fna: The input unannotated FASTA file containing the reference genome</pre>

Step c. Core Genome Alignment using Parsnp

This step aligns the downloaded genomes against the reference genome (from the previous step) to identify core genomic regions and SNPs using Parsnp (<https://github.com/marbl/parsnp>) (Kille et al. 2024)

Command	Description
<pre>#!/bin/bash # Load the Parsnp module module load parsnp/1.5.6</pre>	<pre># parsnp: Calls the Parsnp alignment tool. # -r 91-118.fna: Specifies the path and file for the reference genome in FASTA format. # -g 91-118.gbk: Specifies the path and file for the reference annotation in GenBank format (for mapping SNPs to genes).</pre>

<pre># Execute core genome alignment using the annotated reference from Step 1 parsnp \ -r 91-118.fna \ -g 91-118.gbk \ -d ./fna/*.fna \ -o Xp_parsnp_output \ -p 30</pre>	<pre># -d /scratch/.../*.fna: The directory and file pattern for the query genomes you are aligning against the reference. # -o Parsnp_output: The directory where Parsnp will save its output files. # -p 30: to allocate 30 CPU threads/cores to speed up the process.</pre>
--	--

Step d. Extracting SNPs with HarvestTools

After Parsnp finishes, you use HarvestTools to extract the alignment data into a multi-FASTA format containing only the single-nucleotide polymorphisms (SNPs), which is useful for phylogenetic tree building.

Command	Description
<pre>#!/bin/bash # Download the HarvestTools binary from GitHub using wget wget https://github.com/marbl/harvest- tools/releases/download/v1.2/harvesttools- Linux64-v1.2.tar.gz # Extract the downloaded tarball archive tar -xvf harvesttools-Linux64-v1.2.tar.gz # Navigate into the newly extracted directory cd harvesttools-Linux64-v1.2/ # Extract SNPs from the Parsnp output files ./harvesttools \ -x ../Xp_parsnp_output/parsnp.xmfa \ -i ../Xp_parsnp_output/parsnp.ggr \ -M SNPs.fa</pre>	<pre># tar: Tape archive utility used to extract files. # -xvf: Flags to eXtract, show Verbose output, from a specific File. # cd: Change directory. # harvesttools: Calls the HarvestTools software. # -x ../Xp_parsnp_output/parsnp.xmfa: Inputs the extended Multi-FASTA Alignment (.xmfa) file generated by Parsnp. # -i ../Xp_parsnp_output/parsnp.ggr: Inputs the Gingr (.ggr) archive file generated by Parsnp. # -M SNPs.fa: Outputs a multi-FASTA file containing only the core SNPs.</pre>

Step e: Population structure using RhierBaps

To analyze the resulting SNPs.fa file or visualize the data using the RhierBaps (<https://github.com/gtonkinhill/rhierbaps>) (Tonkin-Hill et al. 2018; Cheng et al. 2013). RStudio is required for this particular tool and visualization.

<pre>#### Step 1: Environment Setup & Dependency Installation This block checks, downloads, and initializes all required CRAN and Bioconductor packages needed for sequence parsing, cluster mapping, and advanced phylogenetic grammar. # 1. Install CRAN packages for core data handling, structural plotting, and clustering</pre>
--

```
# ape: sequence/tree parsing; rhierbaps: population clustering; ggplot2/ggnewscale:  
foundational aesthetics  
if (!requireNamespace("ape", quietly = TRUE)) install.packages("ape")  
if (!requireNamespace("rhierbaps", quietly = TRUE)) install.packages("rhierbaps")  
if (!requireNamespace("ggplot2", quietly = TRUE)) install.packages("ggplot2")  
if (!requireNamespace("ggnewscale", quietly = TRUE)) install.packages("ggnewscale")  
  
# 2. Install Bioconductor manager and downstream tree rendering packages  
if (!requireNamespace("BiocManager", quietly = TRUE)) install.packages("BiocManager")  
if (!requireNamespace("ggtree", quietly = TRUE)) BiocManager::install("ggtree")  
if (!requireNamespace("ggtreeExtra", quietly = TRUE)) BiocManager::install("ggtreeExtra")  
  
# Load library environments into the workspace  
library(ape)  
library(rhierbaps)  
library(ggplot2)  
library(ggnewscale)  
library(ggtree)  
library(ggtreeExtra)
```

#Step 2: Optimization via Segregating Sites Filtering: Large whole-genome alignments contain redundant, invariant columns. This chunk isolates only the polymorphic/segregating positions, generating a lightweight SNP matrix to optimize processing time and memory usage.

1. Parse raw core genome alignment file (FASTA format)

```
dna <- read.dna(  
  "/Users/azk0151/Library/CloudStorage/Box-  
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/SNPs.fa",  
  format = "fasta"  
)
```

2. Extract indices of polymorphic columns across strains

```
seg <- seg.sites(dna)
```

3. Subset full matrix to preserve variable coordinates only

```
dna.snp <- dna[, seg]
```

4. Display final alignment footprint (Strains vs Variable SNPs)

```
dim(dna.snp)
```

5. Export high-efficiency variant-only FASTA file

```
write.dna(  
  dna.snp,  
  file = "/Users/azk0151/Library/CloudStorage/Box-  
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/SNPs_variable_only.fa",  
  format = "fasta",  
  nbcol = -1, # Single unbroken text string per row  
  colsep = "" # Suppress base gap separations  
)
```

#Step 3: Hierarchical Bayesian Analysis of Population Structure (hierBAPS)

1. Feed filtered SNP variants file to specialized rhierbaps alignment interpreter

```
snp.matrix <- load_fasta("/Users/azk0151/Library/CloudStorage/Box-  
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/SNPs_variable_only.fa")
```

2. Audit object integrity

```
class(snp.matrix)  
dim(snp.matrix)
```

3. Compute structural subdivisions

max.depth: Search hierarchy levels; n.pops: Upper bound prior for clusters

```
Perforans.hb <- hierBAPS(snp.matrix, max.depth = 2, n.pops = 20, quiet = FALSE)
```

```
# 4. Confirm matrix layout output (Isolate ID mapped to hierarchical level clusters)
head(Perforans.hb$partition.df)
```

```
# 5. Save clustering classifications for alternative tracking matrices
```

```
write.csv(Perforans.hb$partition.df,
          file = "/Users/azk0151/Library/CloudStorage/Box-
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/Xp_hierbaps.csv",
          row.names = FALSE)
```

Step 4: Circular Tree Visual Engineering and Target Genome Selection

```
# 1. Load the core-genome tree file
```

```
tree <- read.tree("/Users/azk0151/Library/CloudStorage/Box-
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/Xp_parsnp_output/parsnp.tree")
```

```
# 2. Extract your curated sub-list of specific genomes to isolate/label
```

```
highlight_list <- read.csv("/Users/azk0151/Library/CloudStorage/Box-
Box/Amanpreet/APS_2026_workshop/Result_files/Parsnp/Rheirbaps/highlights.csv")$label
```

```
# 3. Initialize circular canvas with conditional annotation tracking
```

```
# ifelse checks if tip matches user highlight CSV: embeds heavy arrowhead (\u2794) if TRUE,
blank if FALSE
```

```
p <- ggtree(tree, layout = 'circular') +
  geom_tiplab(aes(label = ifelse(label %in% highlight_list, paste0("\u2794 ", label), "")),
             size = 2.5,      # Scaled for clear font presentation
             align = TRUE,    # Anchors names into a perfect outer circle
             color = "darkred", # Highlights focused strains vividly
             linetype = "dotted", # Track line linking tip to outer text label
             linesize = 0.3,
             offset = 0.02) + # Distance buffer between tree tips and start of labels
```

```
# 4. Inject structural shape pointers directly onto the tips of highlighted nodes
```

```
geom_tippoint(aes(subset = label %in% highlight_list),
              shape = 17,    # Solid triangle token
              color = "darkred",
              size = 2.5)
```

```
# 5. Format and clear metadata structures for categorical mapping
```

```
heatmapData <- read.csv("~/Downloads/Xp_level_1_heirbaps.csv", row.names = 1)
```

```
rn <- rownames(heatmapData)
```

```
heatmapData <- as.data.frame(sapply(heatmapData, as.character))
```

```
rownames(heatmapData) <- rn
```

```
# 6. Define explicit palette color vectors for sequence clusters 0 through 10
```

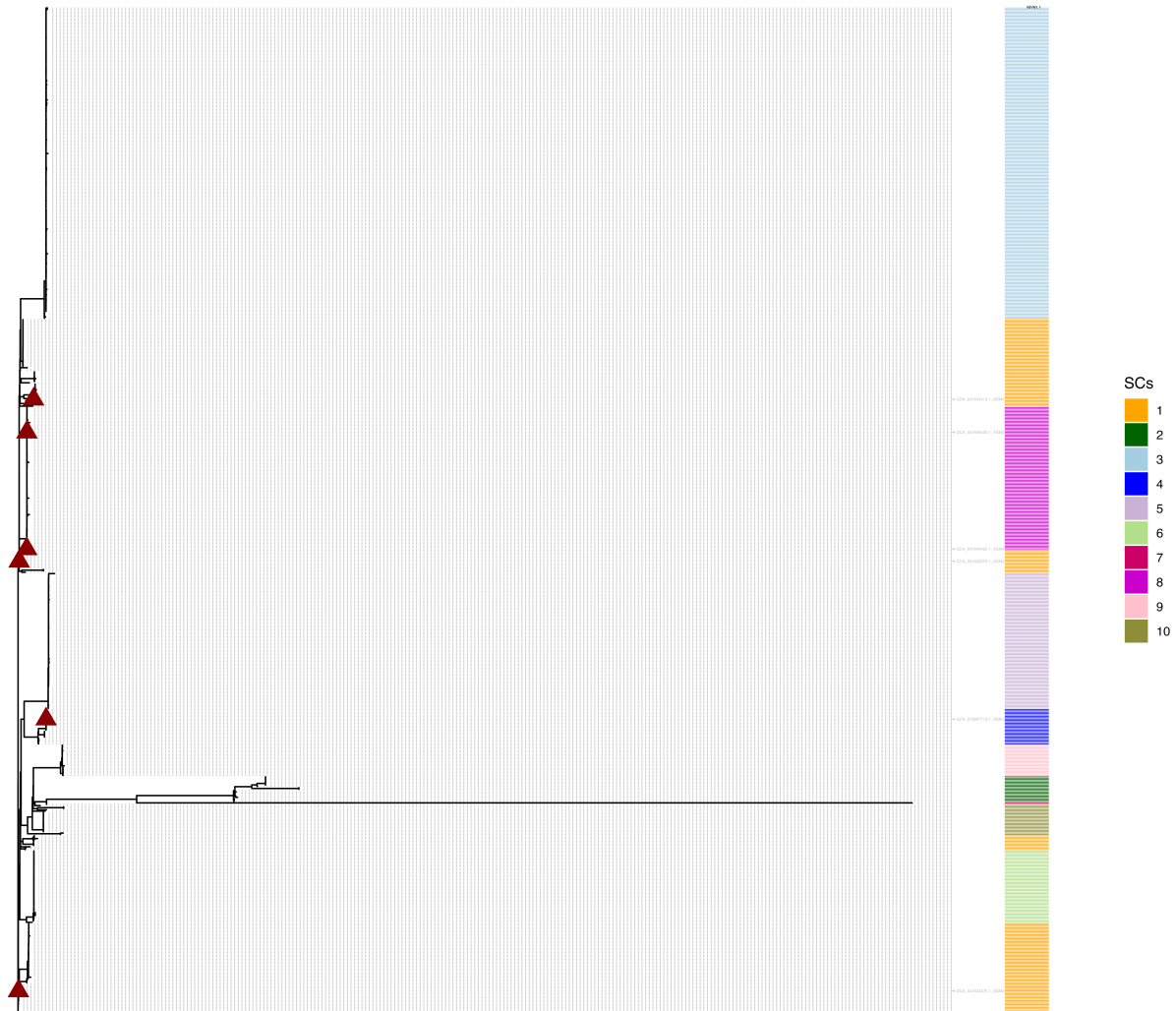
```
heatmap.colours <- c("#ffffff", "orange", "#A6CEE3", "blue", "#CAB2D6",
                    "#B2DF8A", "#CC0066", "#CC00CC", "#FFC0CB", "#8E8E38", "#006400")
names(heatmap.colours) <- 0:10
```

```

# 7. Bind tree model and metadata track matrix seamlessly via gheatmap
# Note: Offset adjusted to accommodate the outer circular text boundaries smoothly
p2 <- gheatmap(p, heatmapData,
  offset = 0.08,      # Distance spacing between text boundary rings and heatmap
  block
    width = 0.06,      # Proportional width of the colored ring track
    font.size = 1.25,
    colnames_position = "top",
    colnames_angle = 0,
    colnames_offset_y = 0,
    hjust = 0) +
  scale_fill_manual(name = "SCs",
    values = heatmap.colours,
    labels = c("1", "2", "3", "4", "5", "6", "7", "8", "9", "10"))

# 8. Compile and present publication-grade figure
p2

```



This figure is generated with the *ggtree* script; the colored triangles represent the different genomes that were used in Part 1A.

Results and Interpretation

Once the *ggtree* script successfully runs, it generates a phylogenetic tree accompanied by colored tracks, which show the genome used in Part 1A. In the output, isolates are grouped numerically; here, we call them **Sequence Clusters (SCs)**. An SC is a discrete grouping defined by the *rhierbaps* Bayesian clustering algorithm. The algorithm scans the core-genome SNP matrix and groups isolates that share highly statistically similar SNP frequencies, assigning them to a distinct partition. In other words, an SC represents a distinct evolutionary lineage or sub-population within the broader species. The core-genome phylogenetic tree was built using *Parsnp*, showing the evolutionary distance between isolates.

Limitations:

- **Reference Genome Bias:** The Parsnp alignment is anchored to a single reference genome. If the reference is basal or distantly related to query genomes, core-genome regions may be underestimated, and SNPs may be mis-assigned due to poor alignment quality. Hence, select a reference genome that is representative of the focal population (e.g., a common lab or clinical isolate). Perform multiple reference-based alignments and compare topologies to ensure consistency. Use reference-free methods (e.g., Mash, Kmer-based clustering) to validate species identification a priori.
- **Limited Sensitivity to Rare Variants:** The core-genome approach focuses on sites variable across multiple genomes. Rare novel variants (having accessory genome differences) are typically excluded, reducing the ability to resolve very recent divergences or within-outbreak microevolution. Augment core-genome phylogenies with accessory-genome analysis. For within-outbreak tracking, supplement with genome-wide SNP counts to capture rare variants.
- **Annotation Quality-Dependent:** Prokka accuracy depends on the quality of training databases and the sequence context. Misannotated genes or incorrect reading-frame assignments can lead to spurious SNP calls at annotation boundaries or in atypical genes. Validate annotations for key genes (virulence factors, resistance genes) using BLAST searches and functional databases (PATRIC, RAST). Use annotation-free SNP extraction (direct Parsnp output) if annotation quality is uncertain.

Acknowledgments

This exercise is part of the APS Plant Health 2026 pre-meeting workshop on 'Genomics and Metagenomics tools and approaches: revealing hidden diversity in agricultural fields. We thank the Auburn University Department of Entomology and Plant Pathology, the Alabama Supercomputer Authority for computational resources, and the APS Plant Health community for feedback and support.

References

- Bhat, J. A., Ali, S., Salgotra, R. K., Mir, Z. A., Dutta, S., Jadon, V., Tyagi, A., Mushtaq, M., Jain, N., Singh, P. K., Singh, G. P., and Prabhu, K. V. 2016. Genomic selection in the era of next generation sequencing for complex traits in plant breeding. *Front. Genet.* 7:221. <https://doi.org/10.3389/fgene.2016.00221>
- Bollas, A., et al. 2025. Variant calling from RNA-Seq data reveals allele-specific differential expression of pathogenic cancer variants. *Commun. Med.* 5:202. <https://doi.org/10.1038/s43856-025-00901-y>
- Campitelli, E. 2024. ggnewscale: Multiple fill and colour scales in 'ggplot2'. R package. <https://CRAN.R-project.org/package=ggnewscale>

- Cheng, L., Connor, T. R., Siren, J., Aanensen, D. M., and Corander, J. 2013. Hierarchical and spatially explicit clustering of DNA sequences with BAPS software. *Mol. Biol. Evol.* 30:1224-1228. <https://doi.org/10.1093/molbev/mst028>
- Danecek, P., Auton, A., Abecasis, G., Albers, C. A., Banks, E., DePristo, M. A., Handsaker, R. E., Lunter, G., Marth, G. T., Sherry, S. T., McVean, G., Durbin, R., and 1000 Genomes Project Analysis Group. 2011. The variant call format and VCFtools. *Bioinformatics* 27:2156-2158. <https://doi.org/10.1093/bioinformatics/btr330>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., and Li, H. 2021. Twelve years of SAMtools and BCFtools. *GigaScience* 10:giab008. <https://doi.org/10.1093/gigascience/giab008>
- Guttman, D. S., Gropp, S. J., Morgan, R. L., and Wang, P. W. 2006. Diversifying selection drives the evolution of the type III secretion system pilus of *Pseudomonas syringae*. *Mol. Biol. Evol.* 23:2342-2354. <https://doi.org/10.1093/molbev/msl103>
- Holloway, B., Luck, S., Beatty, M., Rafalski, J.-A., and Li, B. 2011. Genome-wide expression quantitative trait loci (eQTL) analysis in maize. *BMC Genomics* 12:336. <https://doi.org/10.1186/1471-2164-12-336>
- Kille, B., Nute, M. G., Huang, V., Kim, E., Phillippy, A. M., and Treangen, T. J. 2024. Parsnp 2.0: Scalable core-genome alignment for massive microbial datasets. *Bioinformatics* 40:btac311. <https://doi.org/10.1093/bioinformatics/btac311>
- Li, H. 2011. A statistical framework for SNP calling, mutation discovery, association mapping and population genetical parameter estimation from sequencing data. *Bioinformatics* 27:2987-2993. <https://doi.org/10.1093/bioinformatics/btr509>
- Li, H., and Durbin, R. 2009. Fast and accurate short read alignment with Burrows-Wheeler transform. *Bioinformatics* 25:1754-1760. <https://doi.org/10.1093/bioinformatics/btp324>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., and 1000 Genome Project Data Processing Subgroup. 2009. The Sequence Alignment/Map format and SAMtools. *Bioinformatics* 25:2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>
- McKenna, A., Hanna, M., Banks, E., Sivachenko, A., Cibulskis, K., Kernytsky, A., Garimella, K., Altshuler, D., Gabriel, S., Daly, M., and DePristo, M. A. 2010. The Genome Analysis Toolkit: A MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Res.* 20:1297-1303. <https://doi.org/10.1101/gr.107524.110>
- Newberry, E. A., Minsavage, G. V., Holland, A., Jones, J. B., and Potnis, N. 2023. Genome-wide association to study the host-specificity determinants of *Xanthomonas perforans*. *Phytopathology* 113:400-412. <https://doi.org/10.1094/PHYTO-08-22-0294-R>
- O'Leary, N. A., Cox, E., Holmes, J. B., Anderson, W. R., Falk, R., Hem, V., Tsuchiya, M. T. N., Schuler, G. D., Zhang, X., Torcivia, J., Ketter, A., Breen, L., Cothran, J., Bajwa, H., Tinne, J., Meric, P. A., Hlavina, W., and Schneider, V. A. 2024. Exploring and retrieving sequence and metadata for species across the tree of life with NCBI Datasets. *Sci. Data* 11:732. <https://doi.org/10.1038/s41597-024-03571-y>
- Paradis, E., and Schliep, K. 2019. ape 5.0: An environment for modern phylogenetics and evolutionary analyses in R. *Bioinformatics* 35:526-528. <https://doi.org/10.1093/bioinformatics/bty633>
- Picard Toolkit. 2019. Broad Institute, GitHub Repository. <http://broadinstitute.github.io/picard/>

- R Core Team. 2024. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>
- Robinson, J. T., Thorvaldsdottir, H., Winckler, W., Guttman, M., Lander, E. S., Getz, G., and Mesirov, J. P. 2011. Integrative Genomics Viewer. *Nat. Biotechnol.* 29:24-26. <https://doi.org/10.1038/nbt.1754>
- Seemann, T. 2014. Prokka: Rapid prokaryotic genome annotation. *Bioinformatics* 30:2068-2069. <https://doi.org/10.1093/bioinformatics/btu153>
- Thorvaldsdottir, H., Robinson, J. T., and Mesirov, J. P. 2013. Integrative Genomics Viewer (IGV): High-performance genomics data visualization and exploration. *Brief. Bioinform.* 14:178-192. <https://doi.org/10.1093/bib/bbs017>
- Tonkin-Hill, G., Lees, J. A., Bentley, S. D., Frost, S. D. W., and Corander, J. 2018. RhierBAPS: An R implementation of the population clustering algorithm hierBAPS. *Wellcome Open Res.* 3:93. <https://doi.org/10.12688/wellcomeopenres.14694.1>
- Treangen, T. J., Ondov, B. D., Koren, S., and Phillippy, A. M. 2014. The Harvest suite for rapid core-genome alignment and visualization of thousands of intraspecific microbial genomes. *Genome Biol.* 15:524. <https://doi.org/10.1186/s13059-014-0524-x>
- Wickham, H. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag, New York.
- Xu, S., Dai, Z., Guo, P., Fu, X., Liu, S., Zhou, L., Tang, W., Feng, T., Chen, M., Zhan, L., Wu, T., Hu, E., Jiang, Y., Bo, X., and Yu, G. 2021. ggtreeExtra: Compact visualization of richly annotated phylogenetic data. *Mol. Biol. Evol.* 38:4039-4042. <https://doi.org/10.1093/molbev/msab166>
- Yu, G., Smith, D. K., Zhu, H., Guan, Y., and Lam, T. T.-Y. 2017. ggtree: An R package for visualization and annotation of phylogenetic trees with their covariates and other associated data. *Methods Ecol. Evol.* 8:28-36. <https://doi.org/10.1111/2041-210X.12628>